Kubernetes On-Premises: Architecture and Setup Guide

Oct 16, 2026 · @sachin dubey

On-prem Kubernetes runs on servers you own: bare metal or virtual machines (VMs) in your data center, with containers running your apps on top. This guide explains the architecture, virtualization versus containerization, and a step-by-step kubeadm setup with every command.

1. What does On-Prem mean?

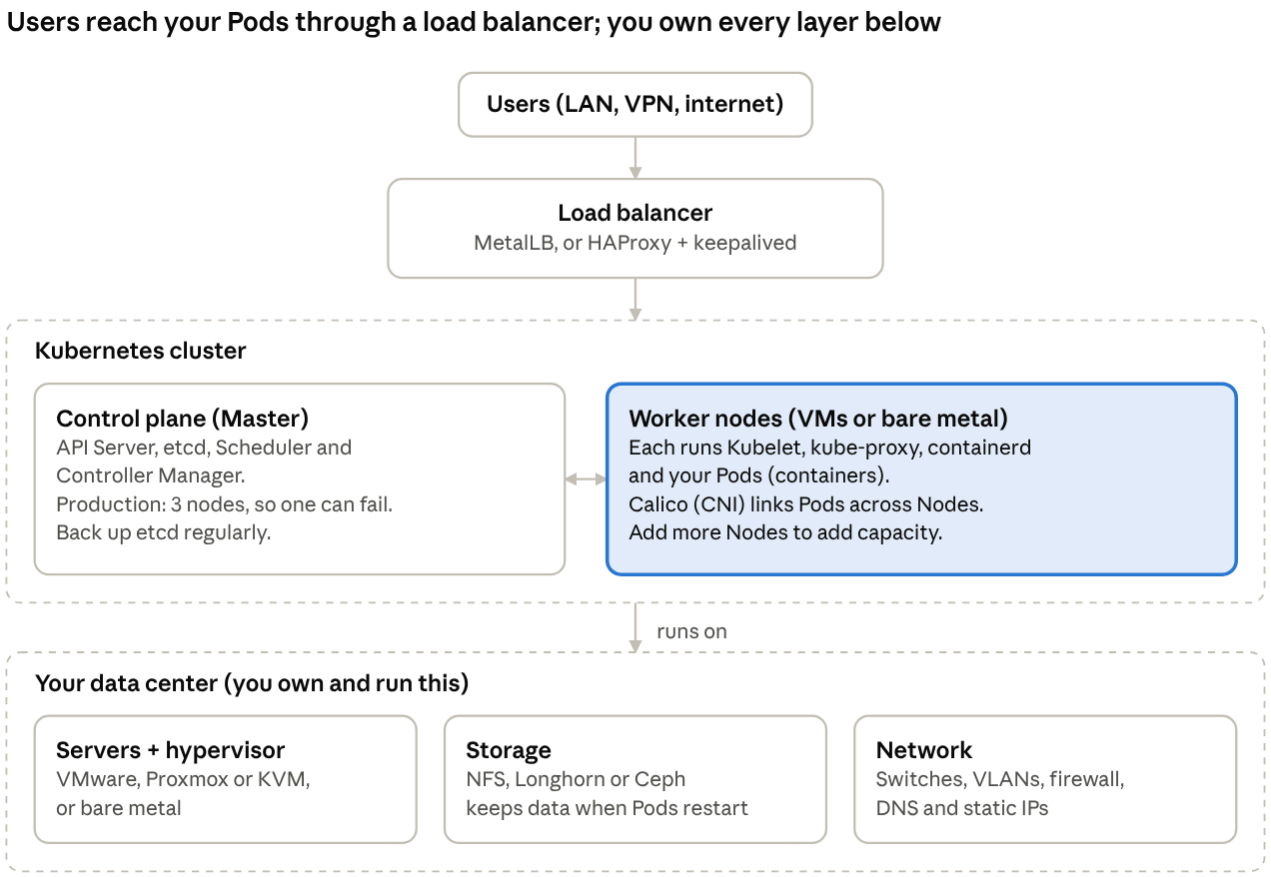
On-premises (on-prem) means the servers, network and storage are in your own office or data center, not rented from a cloud such as AWS, Azure or GCP.

The Kubernetes parts are the same as in the cloud (API Server, etcd, Scheduler, Controller Manager, Kubelet, kube-proxy). What changes is that **you** must provide things the cloud gives for free:

- The machines (physical servers or VMs)
- A load balancer for users and for the API Server
- Persistent storage for data
- Networking, firewall and DNS

2. On-Prem Architecture

The cluster sits on your own servers. Users enter through a load balancer, the control plane makes decisions, worker nodes run the Pods, and your own storage and network sit underneath.



On-prem Kubernetes architecture · 5 layers

Traffic flows down from users to the load balancer and the cluster; everything the cluster runs on is yours to provide.

Layers explained

Layer	What it is	On-prem choice
Users	People or apps that call your service	Office network, VPN or internet
Load balancer	Spreads traffic across Nodes and gives a stable IP	MetalLB, or HAProxy plus keepalived
Control plane (Master)	API Server, etcd, Scheduler, Controller Manager	1 node for a lab, 3 nodes for production
Worker nodes	Run your Pods with Kubelet, kube-proxy and containerd	VMs or bare metal servers
Pod network (CNI)	Lets Pods on different Nodes talk	Calico, Flannel or Weave Net

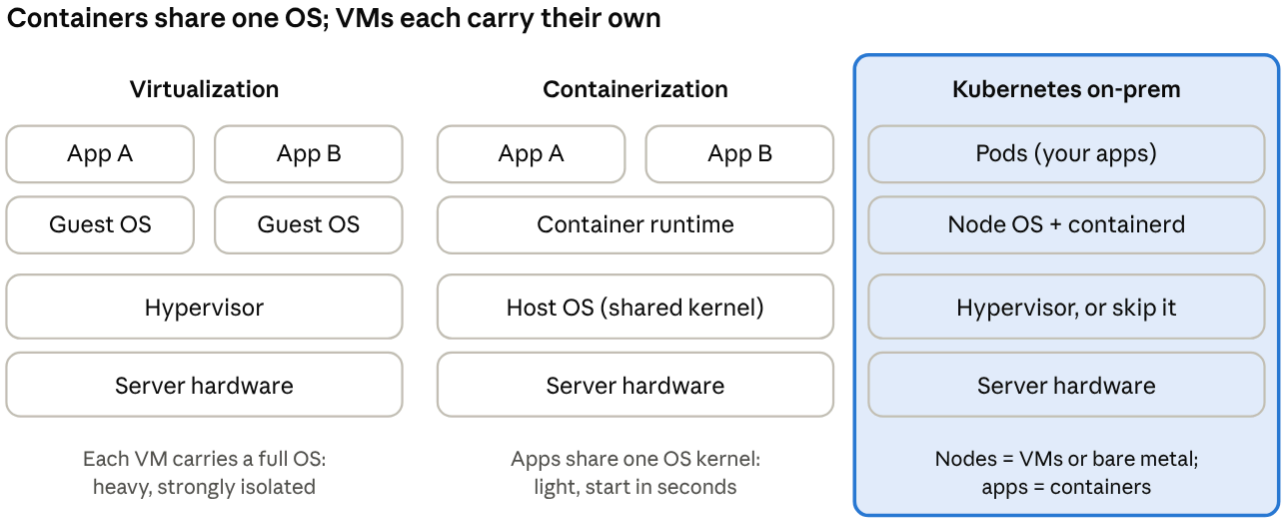
Layer	What it is	On-prem choice
Storage	Keeps data when Pods restart	NFS, Longhorn, Ceph
Physical layer	Servers, switches, power, hypervisor	VMware, Proxmox, KVM, or bare metal

3. Virtualization vs Containerization

These are two different ways to share one physical server between many workloads.

Virtualization uses a **hypervisor** to split one server into several virtual machines. Each VM has its own full operating system, so it is heavy (GBs of disk, minutes to boot) but strongly isolated. Examples: VMware vSphere, Proxmox, KVM, Hyper-V.

Containerization packages an app with its libraries and runs it on the **host's shared OS kernel**. Containers are light (MBs, start in seconds) but share the kernel. Examples: Docker, containerd, CRI-O.



Virtualization vs containerization vs Kubernetes on-prem

On-prem Kubernetes stacks both: an optional hypervisor underneath, containers on top.

Comparison

Point	Virtualization (VM)	Containerization (Container)
Isolation	Own OS and kernel per VM	Shared host kernel
Size	GBs	MBs

Point	Virtualization (VM)	Containerization (Container)
Startup time	Minutes	Seconds
Performance	Some overhead from hypervisor and guest OS	Near native
Density per server	Tens of VMs	Hundreds of containers
Managed by	Hypervisor (vCenter, Proxmox)	Kubernetes
Best for	Running full OS, isolating teams, hosting the Nodes	Packaging and scaling apps

Which one do we use in an On-Prem setup?

Both, at different layers.

- **Containerization is always used.** Kubernetes exists to run containers, so your apps run as containers (via containerd) inside Pods.
- **Virtualization is optional and sits underneath.** The Nodes themselves can be **VMs** (most common, because they are easy to create, snapshot and resize) or **bare metal servers** (best raw performance, used for large or latency-sensitive clusters).

Simple rule: for labs, learning and small or medium companies, use VMs on VMware, Proxmox or KVM as your Nodes. For heavy workloads such as databases, AI or big data, consider bare metal Nodes. In both cases the apps are containers.

4. Example Cluster Used in This Guide

This guide uses three Ubuntu 22.04 or 24.04 machines (VMs or physical). Change the IP addresses to match your network.

Hostname	Role	IP address	CPU	RAM
k8s-master	Control plane	192.168.1.10	2 or more	2 GB or more (4 GB better)
k8s-worker1	Worker	192.168.1.11	2	2 GB or more
k8s-worker2	Worker	192.168.1.12	2	2 GB or more

Requirements: static IPs, full network connectivity between the machines, unique hostnames, internet access to download packages, and a user with `sudo` rights.

5. Step-by-Step Setup with kubeadm

`kubeadm` is the official tool to build a cluster. The steps in **Part A run on ALL nodes**, then **Part B runs on the master only**, then **Part C runs on the workers**.

Part A: Prepare every node (master and workers)

Step 1. Set a hostname and name resolution. Run the matching hostname command on each machine, then add the host entries on all of them.

```
# On the master
sudo hostnamectl set-hostname k8s-master
# On worker 1
sudo hostnamectl set-hostname k8s-worker1
# On worker 2
sudo hostnamectl set-hostname k8s-worker2

# On ALL nodes
cat <<EOF | sudo tee -a /etc/hosts
192.168.1.10 k8s-master
192.168.1.11 k8s-worker1
192.168.1.12 k8s-worker2
EOF
```

Step 2. Turn off swap. Kubelet refuses to run while swap is on.

```
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

Step 3. Load kernel modules and network settings. These let containers use the bridge network and let the node forward packets.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
```

Step 4. Install the container runtime (containerd). This is the engine that actually runs containers.

```
sudo apt-get update
sudo apt-get install -y containerd

sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml

# Use systemd cgroups (required by kubelet)
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml

sudo systemctl restart containerd
sudo systemctl enable containerd
```

Step 5. Install kubeadm, kubelet and kubectl. The version in the URL (`v1.33` here) is the Kubernetes minor version; check kubernetes.io for the release you want and use the same one on every node.

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.33/deb/Release.key | sudo gpg
--dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.33/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```

Part B: Create the control plane (master only)

Step 6. Initialize the cluster. Use the master's own IP. The pod CIDR below matches Calico's default.

```
sudo kubeadm init \
  --apiserver-advertise-address=192.168.1.10 \
  --pod-network-cidr=192.168.0.0/16
```

When it finishes it prints a `kubeadm join ...` command. Copy it.

Step 7. Let your user run kubectl.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

kubectl get nodes
```

The master shows `NotReady` until a network plugin is installed.

Step 8. Install the Pod network (Calico). Check the Calico docs for the current version number and replace `v3.29.3` if a newer one exists.

```
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.29.3/manifests/tige
ra-operator.yaml
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.29.3/manifests/cust
om-resources.yaml

# Wait until all pods are Running
watch kubectl get pods -n calico-system
```

Part C: Join the workers (each worker)

Step 9. Get the join command. If you lost it, or the token expired (tokens last 24 hours), create a new one on the master:

```
kubeadm token create --print-join-command
```

Step 10. Run the join command on each worker. Use your own token and hash from the output above.

```
sudo kubeadm join 192.168.1.10:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Part D: Verify the cluster (master)

Step 11. Check nodes and system pods.

```
kubectl get nodes -o wide
kubectl get pods -A
```

All nodes should show `Ready`. Optionally label the workers:

```
kubectl label node k8s-worker1 node-role.kubernetes.io/worker=
kubectl label node k8s-worker2 node-role.kubernetes.io/worker=
```

Step 12. Deploy a test app.

```
kubectl create deployment nginx --image=nginx --replicas=2
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get pods -o wide
kubectl get svc nginx
```

The service shows a port such as `80:31234/TCP`. Open `http://192.168.1.11:31234` (use your own port) in a browser. The nginx welcome page means your cluster works.

6. On-Prem Extras You Must Plan

Load balancer for Services (MetalLB)

On-prem there is no cloud load balancer, so `type: LoadBalancer` Services stay `Pending`. **MetalLB** hands out IPs from a range you reserve on your network.

```
kubectl apply -f
https://raw.githubusercontent.com/metallb/metallb/v0.14.9/config/manifests/me
tallb-native.yaml
```

Create `metallb-config.yaml` with a free IP range from your LAN, then apply it:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: lan-pool
  namespace: metallb-system
spec:
  addresses:
  - 192.168.1.200-192.168.1.220
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: lan-adv
  namespace: metallb-system
```

```
kubectl apply -f metallb-config.yaml
```

Persistent storage

Pods lose local data when they restart, so use shared storage for databases and uploads. Common on-prem choices:

- **NFS** with an NFS provisioner: simplest, good for labs and small setups
- **Longhorn**: easy replicated storage using the Nodes' own disks
- **Ceph (Rook)**: powerful and scalable, but more complex

High availability for production

One master is a single point of failure. For production use **3 control-plane nodes** behind a virtual IP provided by **HAProxy plus keepalived**, and initialize with that address:

```
sudo kubeadm init \
  --control-plane-endpoint=192.168.1.100:6443 \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16
```

Back up etcd regularly; it holds the whole cluster state.

Firewall ports to open between nodes

Node type	Port	Used for
Control plane	6443	API Server (kubectl and nodes)
Control plane	2379-2380	etcd
Control plane	10257, 10259	Controller Manager, Scheduler
All nodes	10250	Kubelet API
Workers	10256	kube-proxy
Workers	30000-32767	NodePort Services

7. Common Problems

Symptom	Likely cause	Fix
Node stays `NotReady`	No Pod network installed, or Calico pods not running	Complete Step 8 and check `kubectl get pods -n calico-system`
`kubeadm init` fails on preflight	Swap on, or required ports in use	Repeat Step 2, run `sudo kubeadm reset` and try again
Kubelet keeps restarting	containerd not using systemd cgroups	Repeat Step 4 and run `sudo systemctl restart containerd kubelet`
Join command says token invalid	Token older than 24 hours	Run `kubeadm token create --print-join-command` on the master
Service stays `Pending` on LoadBalancer	No load balancer on-prem	Install MetalLB

For logs use `journalctl -u kubelet -f` and `kubectl describe pod <name>`.

8. Command Cheat Sheet

Task	Command
List nodes	`kubectl get nodes -o wide`
List all pods	`kubectl get pods -A`
Describe a resource	`kubectl describe pod <name>`
See logs	`kubectl logs <pod-name>`
Apply a YAML file	`kubectl apply -f file.yaml`
Scale an app	`kubectl scale deployment nginx --replicas=4`
Delete an app	`kubectl delete deployment nginx`
Reset a node	`sudo kubeadm reset`